# HazBot: An Embedded Mobile Manipulation Architecture for Hazardous-Object Retrieval

Anonymous Authors

***Abstract*—Retrieving an object from a potentially hazardous area requires coordinated perception, mobility, and manipulation. We present HazBot, a modular mobile-manipulation architecture centered on a Raspberry Pi, with hazardous-object retrieval as its application objective. Four subsystems provide LiDAR navigation, differential wheel drive, camera-based onboard inference, and an articulated gripper. The camera connects directly to the Pi; dedicated Arduino controllers serve the wheels and arm, while the LiDAR retains its sensor-specific controller. We formalize a seven-phase retrieval mission and the evidence required at each subsystem handoff. A ball-based nonhazardous surrogate motivates the task; evaluation uses an executable discrete coordination model, source-isolated command analysis, and URDF-derived geometry. Exhaustive characterization covers 261,121 wheel-command pairs and reveals direction-only magnitude reduction and 1,020 mixed-zero inputs that emit no new command. The URDF link lengths yield a 0.200 m shoulder-to-wrist bound; 68,921 joint samples characterize the modeled geometric envelope. The full reference coordinator admits no command-contract violations across 28,672 phase–observation pairs. These results connect the mission design to concrete interface and reachability constraints. The contribution is a reproducible architecture and coordination study that supports task-specific recognition and subsequent application validation.**

***Index Terms*—Mobile manipulation, Raspberry Pi, hazardous-object retrieval, embedded vision, subsystem coordination, model-based verification.**

## I. Introduction

Moving a suspect object away from people combines several robotics problems: locating a candidate, navigating toward it, positioning a gripper, transporting the object, and confirming its release. Each component can function in isolation while the complete task remains unreliable. A valid detection may arrive after its target has moved; a motion command may lose its intended magnitude at a serial gateway; an object may lie outside the arm's contact workspace even when the base has reached its navigation goal. The central engineering problem is therefore the relationship between subsystem evidence and permission to act.

Mobile robots have been developed for nuclear environments [1], and compact mobile manipulators demonstrate the value of integrating mobility and reach [2]. HazBot addresses an accessible embedded implementation of this integration. Its central Raspberry Pi connects four functional subsystems: navigation, wheel control, visual inference, and robotic manipulation. Local perception keeps image interpretation on the robot, while dedicated embedded controllers separate sensor acquisition and actuator signaling from mission-level decisions.

We frame the application as retrieval of designated objects from a potentially hazardous area. A task specification determines which objects may be handled and where they should be deposited. Visual recognition proposes a candidate; authorization determines whether handling is permitted. This distinction accommodates task-specific detectors without treating appearance as evidence of chemical, radiological, or biological properties. Ball-based simulated control provides a nonhazardous task surrogate for the present study.

The paper makes three contributions. First, it specifies a Pi-centered four-subsystem architecture with explicit physical interfaces and separate drive, arm, and ranging controller paths. Second, it defines a seven-phase reference coordinator with observable prerequisites for mobility–manipulation handoffs. Third, it provides reproducible model and source analyses that connect these prerequisites to wrist geometry and actual command semantics. The evaluation establishes properties of the discrete coordinator and source models, providing a traceable basis for subsequent physical retrieval experiments.

## II. Related Work

### A. Application and Mobile Manipulation

Vega provides a relevant hazardous-environment precedent: Bird *et al.* describe a small ground robot for nuclear decommissioning, with engineering directed toward its operating environment [1]. Its application illustrates the importance of environment-specific qualification. Stretch combines a differential mobile base and compact manipulator, supported by design analysis and physical task demonstrations [2]. HomeRobot treats navigation, recognition, grasping, and placement as a connected mobile-manipulation problem, with distinct simulation and real-world evaluation components [3]. These studies motivate HazBot's complete retrieval sequence and the coordination of its sensing and actuation modules.

### B. Embedded Integration and Coordination

ROS 2 provides a common communication and software framework for distributed robotic components [4]. Nav2 [5] and SLAM Toolbox [6] provide established navigation and mapping capabilities. These components supply reusable interfaces for integrating onboard observations and motion requests. Edge-device detector evaluations further motivate explicit characterization of model and device choices [7], including the configuration used for local inference.

Command arbitration and sensor-dependent stopping are established in systems such as `twist_mux` and Nav2 Collision Monitor [8], [9]. HazBot focuses on the connection

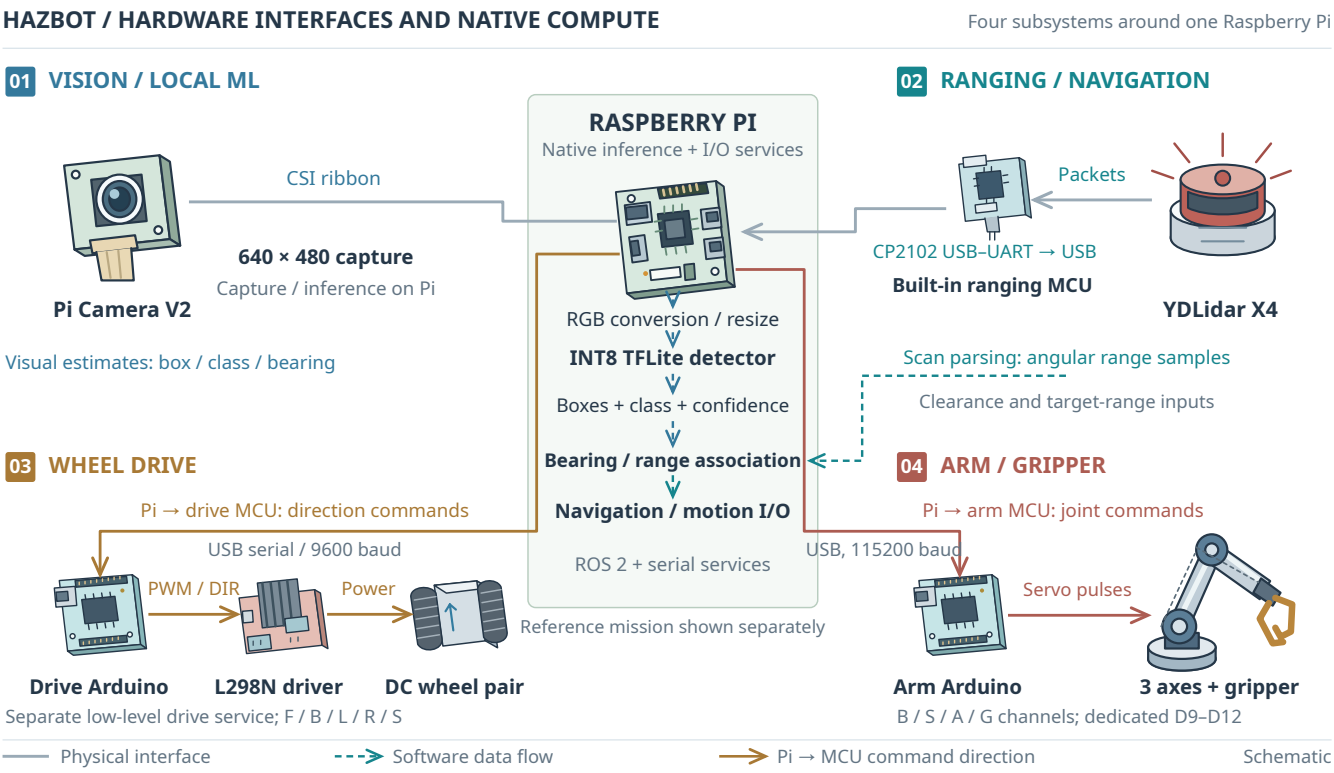



Fig. 1. HazBot's four-subsystem architecture. The Raspberry Pi performs onboard computation and interfaces with a direct CSI camera, a LiDAR sensor controller, a dedicated drive Arduino, and a separate arm Arduino. Solid paths identify physical interfaces; dashed paths show software data flow. Glyphs identify functional hardware rather than a scale assembly. The complete mission coordinator is evaluated as a reference model.

between a four-subsystem embedded design, retrieval-phase prerequisites, and reproducible characterization of the particular geometry and gateways that implement those interfaces. The reference coordinator makes this integration inspectable before a physical mission benchmark is attempted.

## III. System Architecture

### A. Central Computer and Physical Interfaces

Figure 1 places the Raspberry Pi at the computational center. It receives images and range observations, hosts native visual inference, and exchanges commands with the low-level actuator controllers. A workstation may support visualization or development, but is not required to host the native detector. The source configuration analyzed here specifies a Pi Camera V2, YDLIDAR X4 ranging path, differential drive with an L298N bridge, and a servo arm. Table I distinguishes physical connections from logical responsibilities. The physical prototype views in Figs. 2 and 3 identify the components and laptop visualization; its local display is mounted on the Pi through GPIO, and the camera connects by a white CSI ribbon. Figure 4 records the corresponding monitoring and command surfaces without treating a static screenshot as a live-system measurement.

### B. Navigation and Wheel Control

The scanner's built-in controller acquires range samples; its USB interface carries scan packets to the Pi. Navigation logic therefore remains on the central computer rather than on the scanner controller. Range observations support route-clearance decisions and target-distance association. Mapping and navigation components can be connected through ROS 2, but the present evaluation measures neither map quality nor localization error.

TABLE I
Four Subsystems and Their Pi Interfaces

| Subsystem | Connection and responsibility |
|---|---|
| Vision / ML | CSI camera directly to Pi; local TFLite inference. |
| Navigation | LiDAR sensor controller and USB–UART path; ranging data consumed on Pi. |
| Wheel drive | USB serial, 9600 baud, to drive Arduino; PWM/direction through L298N. |
| Manipulator | Separate USB serial, 115200 baud, to arm Arduino; four servo channels. |

The wheel Arduino drives the motor bridge independently of the arm controller. For an ideal differential base, wheel radius $R$ and track $b$ relate desired body velocity $(v, \omega)$ to wheel angular rates:

$$\dot{\phi}_L = \frac{v - b\omega/2}{R}, \qquad \dot{\phi}_R = \frac{v + b\omega/2}{R}. \tag{1}$$

The geometric model uses $R = 0.033\,\mathrm{m}$ and $b = 0.210\,\mathrm{m}$. Equation 1 describes the intended kinematic relationship; a gateway must preserve appropriate command meaning to

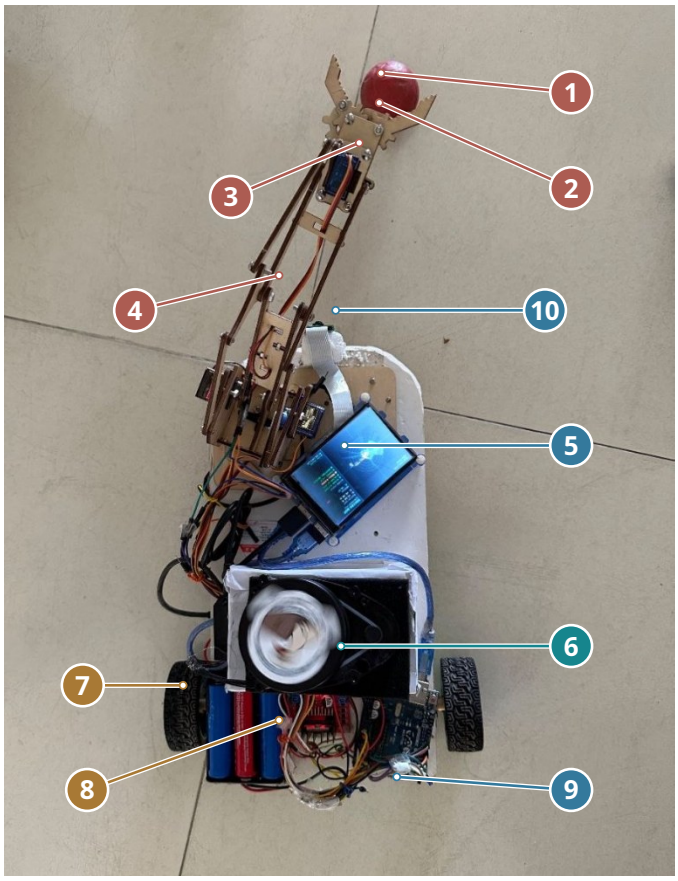



Fig. 2. Physical HazBot prototype. Markers identify: 1, ball surrogate; 2, gripper; 3, servo; 4, arm linkage; 5, Raspberry Pi with GPIO-mounted display; 6, LiDAR; 7, drive wheel; 8, battery cells; 9, control electronics; 10, camera at the end of the white CSI ribbon.

realize it. Our characterization explicitly tests this condition instead of assuming that a pair of numeric inputs produces calibrated wheel velocities.

### C. Vision and Native Inference

The camera acquires RGB images through the Pi's CSI interface. The native path configures a $640 \times 480$ image stream and a two-thread TFLite interpreter for an INT8 detector. Its current label set contains tennis and cricket balls, providing task surrogates. The architecture permits replacing the model and label set, but any extension to designated hazardous objects requires a corresponding dataset and evaluation. Model weights are not part of the analyzed artifacts, so detector accuracy and execution time are outside the reported results.

A detection record should carry its class, confidence, image position, and observation age. Metric approach control additionally requires a range estimate associated with the same target and coordinate frame. Class acceptance and freshness are separate predicates: a confident but stale record cannot authorize a current approach. The task authority supplies the handling policy; visual confidence alone supplies neither material identification nor permission to manipulate an object.

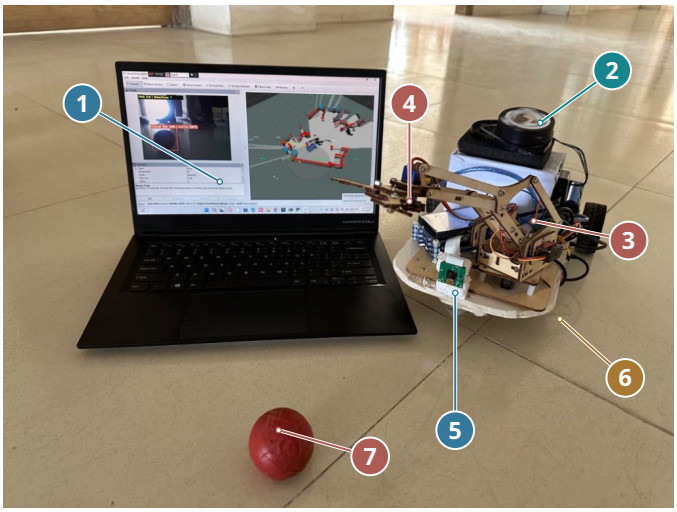



Fig. 3. Front view with laptop visualization. Markers identify: 1, laptop visualization; 2, LiDAR; 3, arm linkage; 4, gripper; 5, CSI camera; 6, mobile base; 7, ball surrogate.

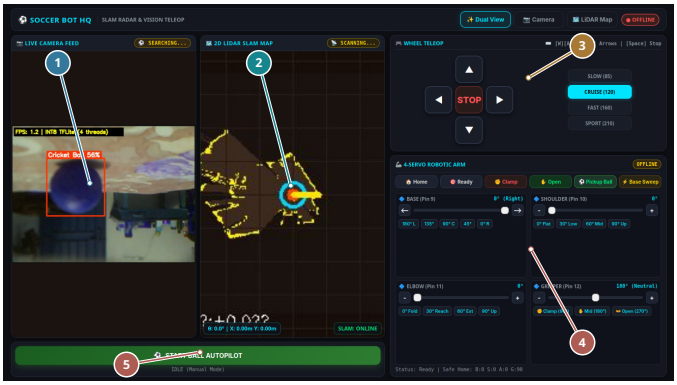



Fig. 4. Annotated operator dashboard supplied with the prototype. Markers identify: 1, camera view and ball-detection panel; 2, LiDAR/SLAM map; 3, wheel teleoperation controls; 4, four-servo arm controls; 5, ball-autopilot action. The screenshot documents the intended monitoring and command surfaces; it is not evidence of online connectivity, detector accuracy, map quality, or autonomous mission success.

### D. Robotic Hand and Arm

The manipulator uses three positioning axes—base yaw, shoulder pitch, and elbow pitch—plus a separately actuated gripper. Its modeled wrist is fixed. The arm Arduino therefore drives four servo channels, rather than four independent positioning joints. Existing arm commands include a preset pickup sequence; the general seven-phase coordinator introduced below is a separate executable reference model. Keeping the drive and arm Arduino paths distinct also avoids conflating their serial protocols and pin assignments.

## IV. Retrieval Coordination and Analysis Method

### A. Mission and Evidence Contracts

The mission consists of SEARCH, APPROACH, DOCK, GRASP, CARRY, PLACE, and COMPLETE (Fig. 5). SEARCH selects an authorized candidate. APPROACH moves

the base toward its pickup region. DOCK transfers control from base motion to positioning the arm. GRASP secures the object, CARRY transports it with the arm stowed, and PLACE releases it at the designated deposit location. COMPLETE closes the task only after a release indication.

Let $T$ denote an authorized, fresh target; $S$ a fresh, usable scan; $C$ a clear route; $A$ a stowed arm; and $B$ a stopped base. The principal command contracts are

$$\text{base motion} \Rightarrow S \wedge C \wedge A, \tag{2}$$

$$\text{target approach} \Rightarrow T, \tag{3}$$

$$\text{arm motion} \Rightarrow B. \tag{4}$$

Reach and gripper closure additionally require $T$ and a reachable wrist target. Release requires the deposit region and a grip indication. The reference controller also requires scan health during docking, grasping, and placement. A failed prerequisite produces HOLD without phase advancement. HOLD means no active-motion request and must be implemented as an explicit stop at an actuator boundary. Logical transitions on target acquisition and release may also emit HOLD. A stop request and a stopped-base observation are distinct events: a deployed implementation needs measured feedback and a defined stop-confirmation rule to instantiate $B$.

The abstract observations comprise twelve Boolean values: target authorization and freshness; scan freshness and usability; route clearance; arm stowage; base stopped; wrist reachability; pickup-region arrival; grip confirmation; deposit-region arrival; and release confirmation. Each has a specific role in the interface, but this discrete model does not generate these observations from physical sensors. In particular, grip and release indications are input assumptions, not inferred force or contact measurements.

### B. Bounded Reference-Model Verification

We enumerate every observation valuation at every phase: $7 \times 2^{12} = 28{,}672$ phase–observation pairs per policy. No valuations are pruned. A separate admissibility oracle checks the output action against Eqs. 2–4 and the target, reach, and placement conditions. Breadth-first exploration establishes phase reachability and extracts shortest-prefix counterexamples. Three ablations remove the base–arm interlock, target authorization, or scan-age requirement from the controller while retaining the full oracle. This design tests whether each omitted condition creates a concrete contract violation.

The enumeration includes observation combinations that may be physically inconsistent. Consequently, counts describe the complete Boolean specification domain rather than probabilities or frequencies of robot failure. A nominal abstract trace checks that admissible observations permit progression to COMPLETE. Neither this trace nor phase reachability measures physical mission success.

### C. Source-Derived Wrist Geometry

Figure 6 connects arm geometry to the docking prerequisite. From the URDF chain, the shoulder lies at $(0.100, 0, 0.155)$ m in the base frame and both positioning links have length $L_1 = L_2 = 0.100$ m. The zero pose has a vertical upper link and horizontal forearm. The figure uses $q_1 = q_b$, $q_2 = q_s$, and $q_3 = q_e$. For yaw $q_b$, shoulder angle $q_s$, and elbow angle $q_e$, the wrist origin is

$$r = L_1 \sin q_s + L_2 \cos(q_s + q_e), \tag{5}$$

$$x = 0.100 + r \cos q_b, \quad y = r \sin q_b, \tag{6}$$

$$z = 0.155 + L_1 \cos q_s - L_2 \sin(q_s + q_e). \tag{7}$$

We sample 41 values, including both bounds, for each positioning joint. The declared bounds are $q_b \in [-1.5708, 1.5708]$, $q_s \in [-0.5, 2.6179]$, and $q_e \in [-1.5708, 2.6179]$ rad. An independent homogeneous-transform implementation evaluates the URDF chain at 1,000 seeded random poses and 13 special poses. Agreement checks numerical implementation, while the grid characterizes the ideal wrist envelope. The wrist origin is not a calibrated grasp contact point; collision, load, compliance, and stability constraints are not represented.

### D. Command-Interface Characterization

We isolate the actual wheel gateway's `SET:` branch using Python's abstract syntax tree and replace only its transport sender with an in-memory recorder. All integer pairs $(l, r) \in [-255, 255]^2$ are executed and compared with an independently specified sign partition. The arm gateway's pure angle mapper is evaluated for five accepted aliases over 451 inputs from $-90$ to 360, checking clamps and reversals. The elbow has two aliases, so this represents four actuator channels. No ROS constructors, sockets, motor servers, or hardware launchers are executed. Source hashes, settings, CSV outputs, and validation records bind the results to the analyzed implementation.

## V. Results

### A. Mission Handoffs

The full reference coordinator admits 0 contract-violating commands among 28,672 phase–observation pairs. All 7 phases are reachable, with 13 unique phase edges (including self-loops) and a shortest completion depth of 6 transitions. A separate 11-step nominal witness exercises all 9 action types and reaches COMPLETE.

Removing the base–arm interlock produces 480 violating pairs; removing target authorization produces 128; and removing scan-age checks produces 288. Their shortest counterexample prefixes contain 1, 2, and 1 observations, respectively. For example, the interlock ablation permits SEARCH rotation while the arm is not stowed. These counts demonstrate guard sensitivity over the enumerated domain; they are not observed mission failure rates.

### B. Geometric Envelope and Docking

The $41^3$ grid contains 68,921 wrist configurations. The shoulder-to-wrist distance is bounded by the link sum, $L_1 + L_2 = 0.200$ m. The sampled wrist height spans $-0.0316$ to $0.3549$ m in the base frame. Of the samples, 2,747 lie below the modeled $z = 0$ ground plane and 66,174 lie on

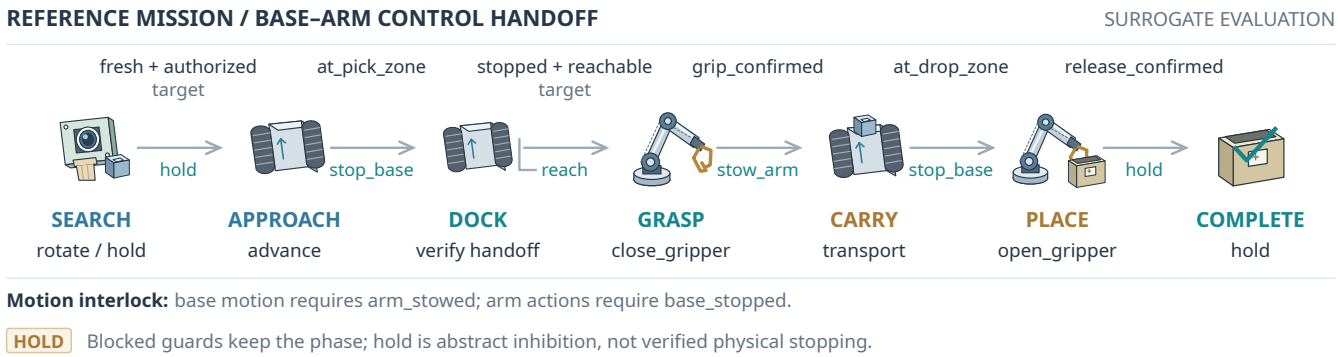



Fig. 5. Seven-phase reference retrieval mission. Conditions summarize the handoff evidence; complete command predicates are defined in the text and executable model. A failed prerequisite requests HOLD at the current phase. The strip depicts the coordination specification, rather than a measured physical trajectory.

TABLE II
BOUNDED REFERENCE-MODEL ABLATIONS

| Reference policy | Violating pairs | Prefix length |
|---|---|---|
| All guards | 0 | — |
| Without base–arm interlock | 480 | 1 |
| Without target authorization | 128 | 2 |
| Without scan age | 288 | 1 |

Each policy evaluates 28,672 pairs. Prefix length counts observations from SEARCH to the first violating output. Counts concern abstract command admissibility.

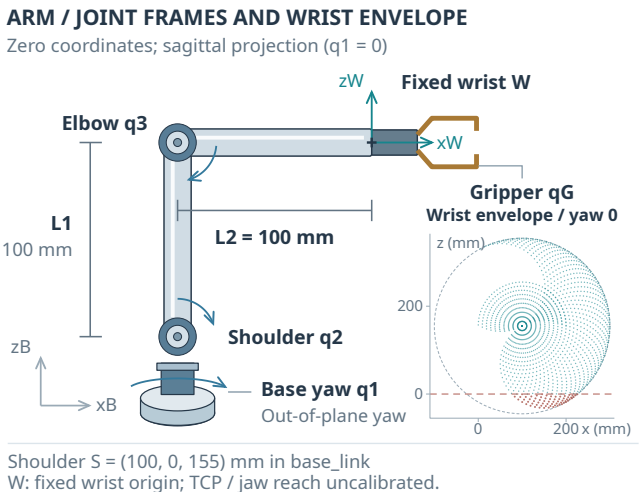



Fig. 6. Source-derived arm geometry at zero joint coordinates, with joint axes and fixed wrist origin. The small inset shows 1,681 zero-yaw wrist samples, the link-sum bound, and modeled ground exclusions. Samples describe ideal geometry rather than calibrated contact positions or collision-free grasps.

or above it. These counts are joint-grid counts, not Cartesian workspace volumes; an above-ground sample may still violate self-collision or stability constraints.

Independent forward-kinematics implementations agree over 1,013 validation poses with maximum positional difference $6.8 \times 10^{-17}$ m. This near-machine-precision agreement supports the analytical implementation, not mechanical accuracy. The actionable implication is that docking must use a calibrated contact workspace and target pose, rather than a generic visual stopping distance. The wrist envelope provides a reproducible geometric starting point for that calibration.

### C. Wheel and Arm Command Semantics

All 261,121 wheel input pairs match the independent sign partition. Each nonzero sign quadrant contains 65,025 pairs and produces one of four direction commands. Magnitudes within a quadrant are discarded. Exactly one pair, $(0, 0)$, emits a stop command; the 1,020 mixed-zero pairs emit no new command. Thus, commanding one stationary wheel and one moving wheel through this branch does not produce the requested wheel pair. No new command must also be distinguished from a stop: the experiment records emitted messages, not subsequent actuator behavior.

This interface behavior directly qualifies Eq. 1. A continuous differential-drive command cannot be assumed to survive a direction-only gateway. Either the upstream controller must deliberately use discrete motion primitives, or the gateway and low-level control must preserve calibrated wheel-rate semantics. This is an integration requirement for controlled approach and docking, not a measured speed error.

The arm mapping matches an independent clamp/reversal specification for all 2,255 evaluated cases. Base commands retain direction, shoulder and elbow conventions are reversed, and the gripper uses its declared command range. These are software command conventions; translating them to the modeled joint angles requires physical calibration. Across geometry, wheel semantics, arm mapping, and schema extraction, all 29 characterization checks pass. The current detector's two named classes confirm the surrogate task configuration; no recognition score is inferred from those labels.

## VI. DISCUSSION AND APPLICATION DEVELOPMENT

The architecture connects object retrieval through explicit subsystem evidence: vision proposes a candidate, task authority permits handling, navigation establishes a pickup region, the base confirms rest, and manipulation confirms grip and release. The mission model exposes these obligations where

TABLE III
REPRODUCIBLE CHARACTERIZATION RESULTS

| Quantity | Computed result |
|---|---|
| Joint-grid wrist poses | 68,921 |
| Modeled shoulder–wrist bound | 0.200 m |
| Poses below modeled ground | 2,747 |
| Independent FK validation poses | 1,013 |
| Exhaustive wheel input pairs | 261,121 |
| Mixed-zero pairs with no command | 1,020 |
| Arm mapping cases / physical channels | 2,255 / 4 |
| Characterization checks passed | 29 / 29 |

hardware feedback remains to be developed. Its nominal trace demonstrates logical progress under stated assumptions, while source analysis identifies what the actuator gateway can express.

The next application step is to instantiate these observations with measured quantities. This requires a task-specific visual dataset and held-out recognition results; calibrated camera–LiDAR–arm frames and contact geometry; feedback for base rest, grip, and release; and timed physical runs covering navigation, pickup, transport, and deposit. Sensor-age limits must follow measured acquisition and transport behavior. Force, payload, containment, and environmental compatibility must be evaluated for the designated object class. A general RGB category detector cannot establish an unknown material's hazardous properties.

The results establish source and model properties; physical obstacle avoidance and hazardous-environment suitability require application trials. Nonhazardous surrogates support the initial benchmark, followed by object- and environment-specific qualification as illustrated by prior systems [1], [2].

## VII. CONCLUSION

HazBot connects navigation, wheel control, native visual inference, and manipulation through a Raspberry Pi-centered architecture. Its seven-phase reference mission formalizes retrieval handoffs, while reproducible geometry and gateway analysis expose docking constraints, magnitude reduction, and mixed-zero command omissions. These artifacts provide an implementation-grounded basis for task-specific hazardous-object retrieval and subsequent physical evaluation.

## ACKNOWLEDGMENT

The authors acknowledge the ROS 2, Nav2, and open-source robotics communities for their publicly available software and technical documentation. ChatGPT, a large language model developed by OpenAI, assisted literature discovery and synthesis, study design, code development, analysis, manuscript drafting, and editing. The authors are responsible for reviewing and approving the content and for the accuracy and integrity of the submitted work.

## REFERENCES

[1] B. Bird, M. Nancekievill, A. West, J. Hayman, C. Ballard, W. Jones, S. Ross, T. Wild, T. Scott, and B. Lennox, "Vega—a small, low cost, ground robot for nuclear decommissioning," *Journal of Field Robotics*, vol. 39, no. 3, pp. 232–245, 2022. [Online]. Available: https://onlinelibrary.wiley.com/doi/full/10.1002/rob.22048

[2] C. C. Kemp, A. Edsinger, H. M. Clever, and B. Matulevich, "The design of Stretch: A compact, lightweight mobile manipulator for indoor human environments," in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3150–3157. [Online]. Available: https://arxiv.org/abs/2109.10892

[3] S. Yenamandra, A. Ramachandran, K. Yadav, A. S. Wang, M. Khanna, T. Gervet, T.-Y. Yang, V. Jain, A. Clegg, J. M. Turner, Z. Kira, M. Savva, A. X. Chang, D. S. Chaplot, D. Batra, R. Mottaghi, Y. Bisk, and C. Paxton, "HomeRobot: Open-vocabulary mobile manipulation," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 1975–2011. [Online]. Available: https://proceedings.mlr.press/v229/yenamandra23a.html

[4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: https://arxiv.org/abs/2211.07752

[5] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2718–2725. [Online]. Available: https://arxiv.org/abs/2003.00368

[6] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021. [Online]. Available: https://joss.theoj.org/papers/10.21105/joss.02783

[7] D. K. Alqahtani, M. A. Cheema, M. A. Rodriguez, and A. N. Toosi, "A comprehensive evaluation of deep learning object detection models on heterogeneous edge devices," arXiv:2409.16808v3, 2026, version 3, July 14, 2026. [Online]. Available: https://arxiv.org/abs/2409.16808v3

[8] ROS Teleop Contributors, "twist_mux: Input topic configuration," Official project source, rolling branch, accessed: Oct. 21, 2026. [Online]. Available: https://github.com/ros-teleop/twist_mux/blob/rolling/config/twist_mux_topics.yaml

[9] Nav2 Project, "Collision monitor node," Nav2 Documentation, accessed: Oct. 21, 2026. [Online]. Available: https://docs.nav2.org/rolling/configuration_and_development/configuration_guide/core_servers/collision_monitor/configuring_collision_monitor_node/